

Step 1: Import Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.ensemble import (
    RandomForestClassifier,
    RandomForestRegressor
)

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    mean_absolute_error,
    r2_score
)

plt.style.use('ggplot')
```

Step 2: Upload Dataset

```
from google.colab import files

uploaded = files.upload()
```

Choose Files | No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving SEO_Dataset_After_v3.csv to SEO_Dataset_After_v3.csv

Step 3: Load Data

```
before = pd.read_csv("SEO_Dataset_Before_v3.csv")
after = pd.read_csv("SEO_Dataset_After_v3.csv")
```

Step 4: Merge Before & After Data

```
data = before.merge(
    after,
    on="URL",
    suffixes=("_before", "_after")
)

print(data.shape)

data.head()
```

(150, 51)

	URL	Size_KB_before	Status Code_before	Indexability_Num_before	Crawl Depth_before	Folder Depth_before	Inlinks
0	https://wooferzz.com/	166.67	200	1	0	0	
1	https://wooferzz.com/blogs/news	79.07	200	1	0	1	
2	https://wooferzz.com/blogs/news/wooferzz- healt...	89.56	200	1	0	2	
3	https://wooferzz.com/collections/biscuits	221.08	200	1	0	1	
4	https://wooferzz.com/collections/bowl	187.74	200	1	0	1	

5 rows × 51 columns

Step 5: Feature Engineering

Create all improvement metrics together.

```
data["Performance_Improvement"] = (
    data["Performance_Score_after"]
    - data["Performance_Score_before"]
)

data["Speed_Improvement"] = (
    data["Response Time_before"]
    - data["Response Time_after"]
)

data["LCP_Improvement"] = (
    data["LCP Score_before"]
    - data["LCP Score_after"]
)

data["SEO_Health_Improvement"] = (
    data["SEO_Health_Score_after"]
    - data["SEO_Health_Score_before"]
)

data["Target"] = (
    data["Performance_Improvement"] > 0
).astype(int)

data.head()
```

	URL	Size_KB_before	Status Code_before	Indexability_Num_before	Crawl Depth_before	Folder Depth_before	Inlinks
0	https://wooferzz.com/	166.67	200		1	0	0
1	https://wooferzz.com/blogs/news	79.07	200		1	0	1
2	https://wooferzz.com/blogs/news/wooferzz-healt...	89.56	200		1	0	2
3	https://wooferzz.com/collections/biscuits	221.08	200		1	0	1
4	https://wooferzz.com/collections/bowl	187.74	200		1	0	1

5 rows × 8 columns

Step 6: Dataset Overview

```
print(data.info())

print(data.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 56 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   URL                                    150 non-null    object
1   Size_KB_before                        150 non-null    float64
2   Status Code_before                    150 non-null    int64
3   Indexability_Num_before               150 non-null    int64
4   Crawl Depth_before                    150 non-null    int64
5   Folder Depth_before                   150 non-null    int64
6   Inlinks_before                        150 non-null    int64
7   Unique Inlinks_before                 150 non-null    int64
8   Outlinks_before                       150 non-null    int64
9   Response Time_before                  150 non-null    float64
10  Word Count_before                     150 non-null    int64
11  FCP_before                            150 non-null    float64
12  CLS_before                            150 non-null    float64
13  LCP Score_before                      150 non-null    int64
14  TBT_Milliseconds_before               150 non-null    int64
15  Speed Index_before                    150 non-null    float64
16  Performance_Score_before              150 non-null    int64
17  SEO_Health_Score_before               150 non-null    int64
18  SEO Status_before                     150 non-null    object
19  Performance Flag_before               150 non-null    object
```

20	Heavy Page Flag_before	150	non-null	object
21	Performance Category_before	150	non-null	object
22	Indexability Clean_before	150	non-null	int64
23	LCP Category_before	150	non-null	object
24	Slow Page Flag_before	150	non-null	object
25	SEO Issue Status_before	150	non-null	object
26	Size_KB_after	150	non-null	float64
27	Status Code_after	150	non-null	int64
28	Indexability_Num_after	150	non-null	int64
29	Crawl Depth_after	150	non-null	int64
30	Folder Depth_after	150	non-null	int64
31	Inlinks_after	150	non-null	int64
32	Unique Inlinks_after	150	non-null	int64
33	Outlinks_after	150	non-null	int64
34	Response Time_after	150	non-null	float64
35	Word Count_after	150	non-null	int64
36	FCP_after	150	non-null	float64
37	CLS_after	150	non-null	float64
38	LCP Score_after	150	non-null	int64
39	TBT_Milliseconds_after	150	non-null	int64
40	Speed Index_after	150	non-null	float64
41	Performance_Score_after	150	non-null	int64
42	SEO_Health_Score_after	150	non-null	int64
43	SEO Status_after	150	non-null	object
44	Performance Flag_after	150	non-null	object
45	Heavy Page Flag_after	150	non-null	object
46	Performance Category_after	150	non-null	object
47	Indexability Clean_after	150	non-null	int64
48	LCP Category_after	150	non-null	object
49	Slow Page Flag_after	150	non-null	object
50	SEO Issue Status_after	150	non-null	object
51	Performance Improvement	150	non-null	int64

Step 7: SEO Status Comparison

```

before_counts = data[
    "SEO Issue Status_before"
].value_counts()

after_counts = data[
    "SEO Issue Status_after"
].value_counts()

comparison = pd.DataFrame({
    "Before": before_counts,
    "After": after_counts
})

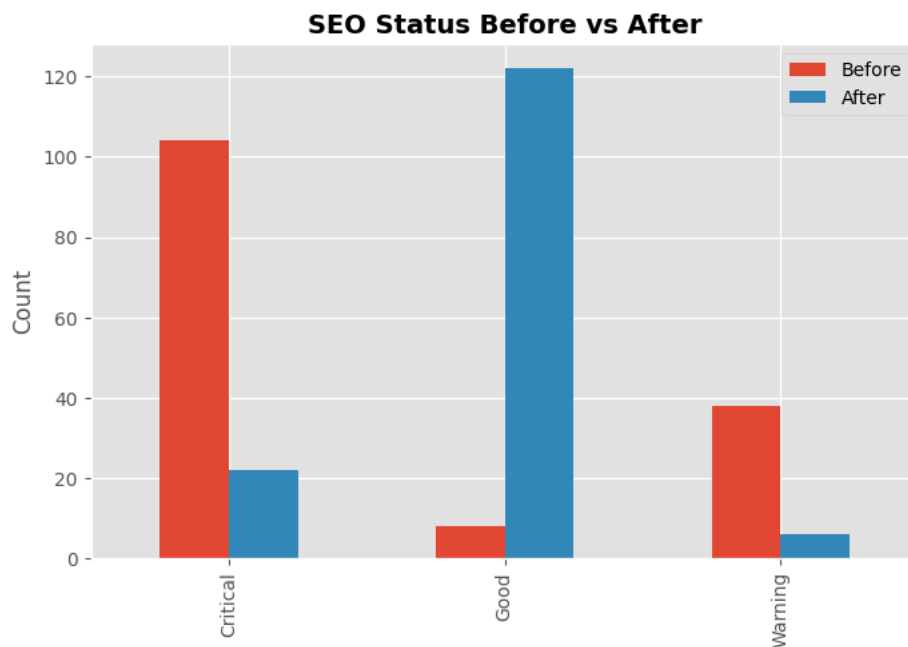
comparison.plot(
    kind="bar",
    figsize=(8,5)
)

plt.title(
    "SEO Status Before vs After",
    fontsize=14,
    fontweight="bold"
)

plt.ylabel("Count")

plt.show()

```



Step 8: Correlation Heatmap

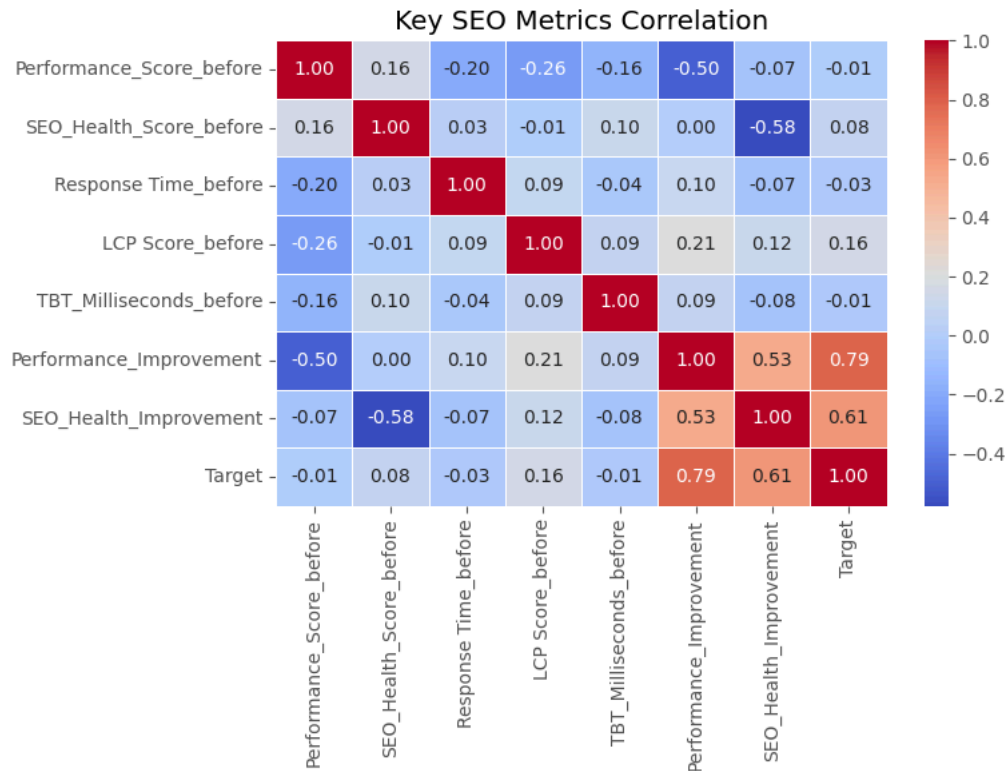
```
important_cols = [
    'Performance_Score_before',
    'SEO_Health_Score_before',
    'Response_Time_before',
    'LCP_Score_before',
    'TBT_Milliseconds_before',
    'Performance_Improvement',
    'SEO_Health_Improvement',
    'Target'
]

corr_matrix = data[important_cols].corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title("Key SEO Metrics Correlation")
plt.tight_layout()
plt.show()
```



Step 9: Performance Improvement Distribution

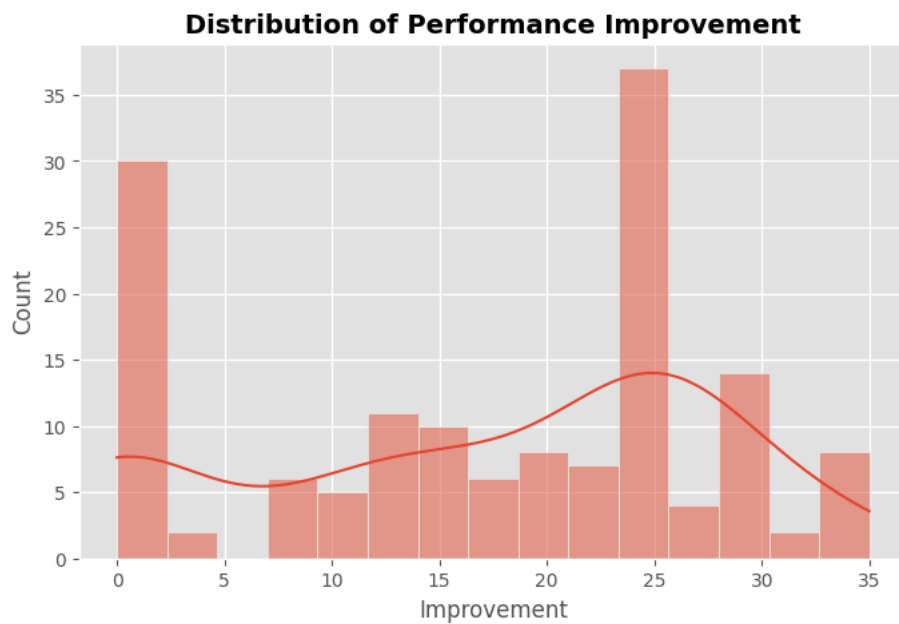
```
plt.figure(figsize=(8,5))

sns.histplot(
    data["Performance_Improvement"],
    bins=15,
    kde=True
)

plt.title(
    "Distribution of Performance Improvement",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Improvement")

plt.show()
```



Step 10: Before vs After Scatter Plot

```
plt.figure(figsize=(8,6))

plt.scatter(
    data["Performance_Score_before"],
    data["Performance_Score_after"],
    alpha=0.7,
    s=80
)

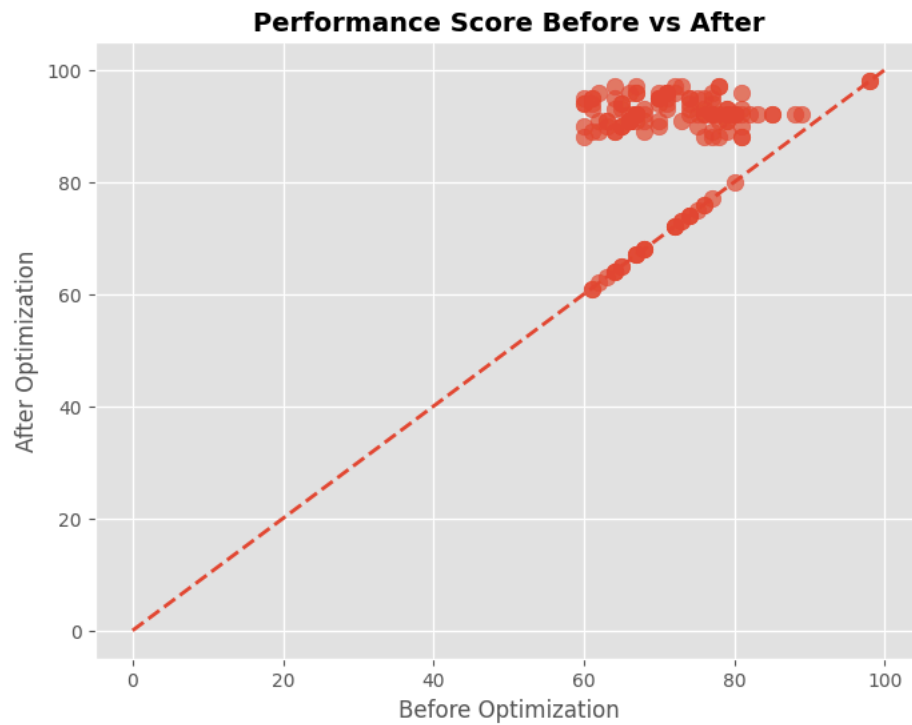
plt.plot(
    [0,100],
    [0,100],
    '--',
    linewidth=2
)

plt.xlabel(
    "Before Optimization"
)

plt.ylabel(
    "After Optimization"
)

plt.title(
    "Performance Score Before vs After",
    fontsize=14,
    fontweight="bold"
)

plt.show()
```



Step 11: Top Improved Pages

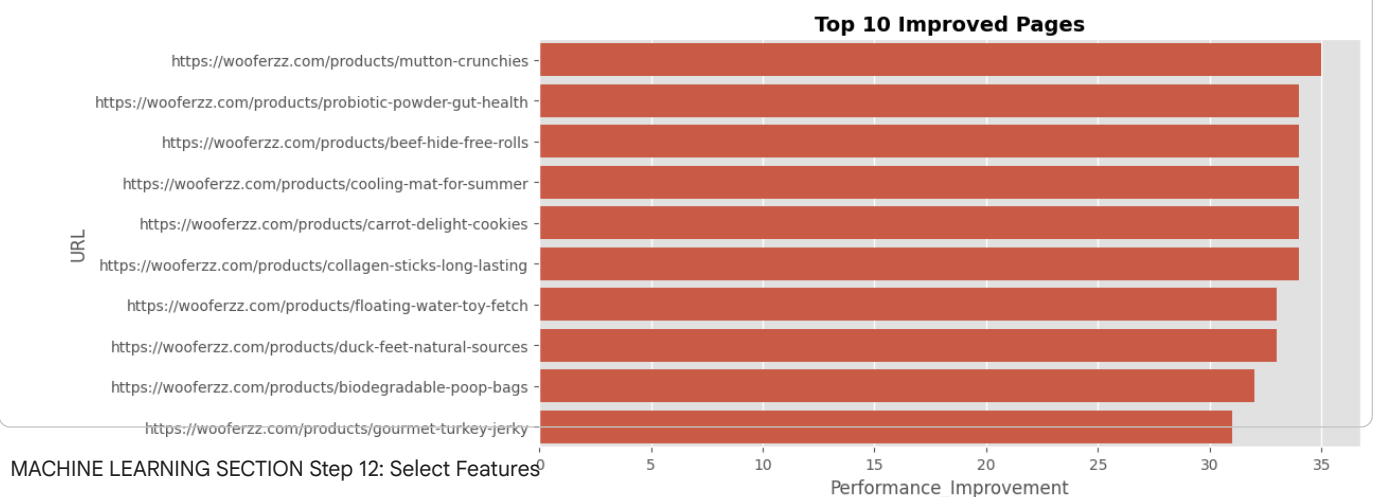
```
top_pages = data.sort_values(
    by="Performance_Improvement",
    ascending=False
).head(10)

plt.figure(figsize=(10,5))

sns.barplot(
    data=top_pages,
    x="Performance_Improvement",
    y="URL"
)

plt.title(
    "Top 10 Improved Pages",
    fontsize=14,
    fontweight="bold"
)

plt.show()
```



```

features = [

    "Performance_Score_before",

    "SEO_Health_Score_before",

    "Response Time_before",

    "LCP Score_before",

    "TBT_Milliseconds_before",

    "Word Count_before"

]

X = data[features]

y = data["Target"]

```

Step 13: Train-Test Split

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

```

Step 14: Random Forest Classification

```

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(
    X_train,
    y_train
)

y_pred = rf_model.predict(
    X_test
)

```

Step 15: Classification Evaluation

```

print(
    "Accuracy:",
    accuracy_score(
        y_test,
        y_pred
    )
)

print(
    classification_report(
        y_test,
        y_pred
    )
)

```

```

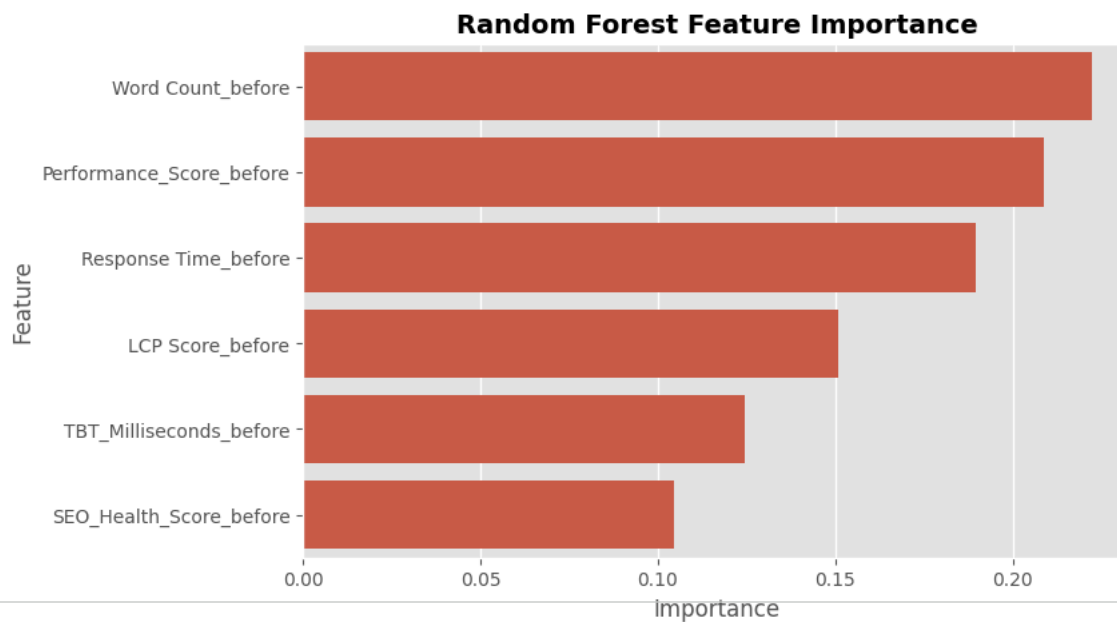
Accuracy: 0.8

```

	precision	recall	f1-score	support
0	1.00	0.14	0.25	7
1	0.79	1.00	0.88	23
accuracy			0.80	30
macro avg	0.90	0.57	0.57	30
weighted avg	0.84	0.80	0.74	30

Step 16: Feature Importance

```
importance = pd.DataFrame({  
    "Feature": features,  
    "Importance":  
        rf_model.feature_importances_  
})  
  
importance = importance.sort_values(  
    by="Importance",  
    ascending=False  
)  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    data=importance,  
    x="Importance",  
    y="Feature"  
)  
  
plt.title(  
    "Random Forest Feature Importance",  
    fontsize=14,  
    fontweight="bold"  
)  
  
plt.show()
```



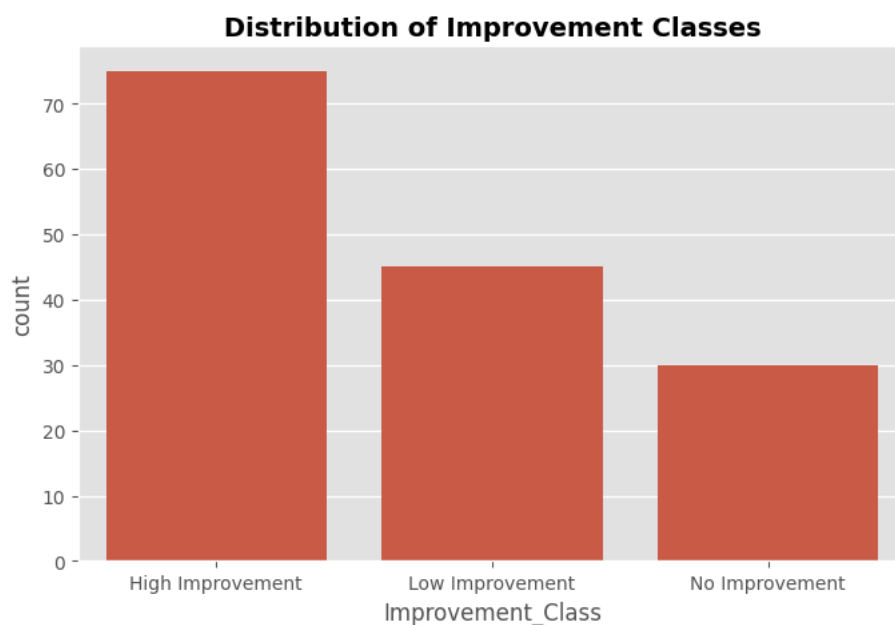
MULTI-CLASS MODEL Step 17: Create Improvement Classes

```
def classify_improvement(x):  
  
    if x >= 20:  
        return "High Improvement"  
  
    elif x > 0:  
        return "Low Improvement"  
  
    else:  
        return "No Improvement"  
  
data["Improvement_Class"] = data[  
    "Performance_Improvement"  
].apply(classify_improvement)  
  
data["Improvement_Class"].value_counts()
```

	count
Improvement_Class	
High Improvement	75
Low Improvement	45
No Improvement	30

dtype: int64

```
plt.figure(figsize=(8,5))  
  
sns.countplot(  
    data=data,  
    x="Improvement_Class"  
)  
  
plt.title(  
    "Distribution of Improvement Classes",  
    fontsize=14,  
    fontweight="bold"  
)  
  
plt.show()
```



Step 18: Train Multi-Class Model

```
y_multi = data[  
    "Improvement_Class"  
]
```

```

X_train_mc, X_test_mc, y_train_mc, y_test_mc = train_test_split(
    X,
    y_multi,
    test_size=0.2,
    random_state=42
)

multi_rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

multi_rf.fit(
    X_train_mc,
    y_train_mc
)

multi_pred = multi_rf.predict(
    X_test_mc
)

```

Evaluation

```

print(
    accuracy_score(
        y_test_mc,
        multi_pred
    )
)

print(
    classification_report(
        y_test_mc,
        multi_pred
    )
)

```

```

0.7

```

	precision	recall	f1-score	support
High Improvement	0.70	0.93	0.80	15
Low Improvement	0.70	0.88	0.78	8
No Improvement	0.00	0.00	0.00	7
accuracy			0.70	30
macro avg	0.47	0.60	0.53	30
weighted avg	0.54	0.70	0.61	30

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

Visual

```

cm_multi = confusion_matrix(
    y_test_mc,
    multi_pred
)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm_multi,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=multi_rf.classes_,
    yticklabels=multi_rf.classes_
)

plt.title(
    "Multi-Class Confusion Matrix"
)

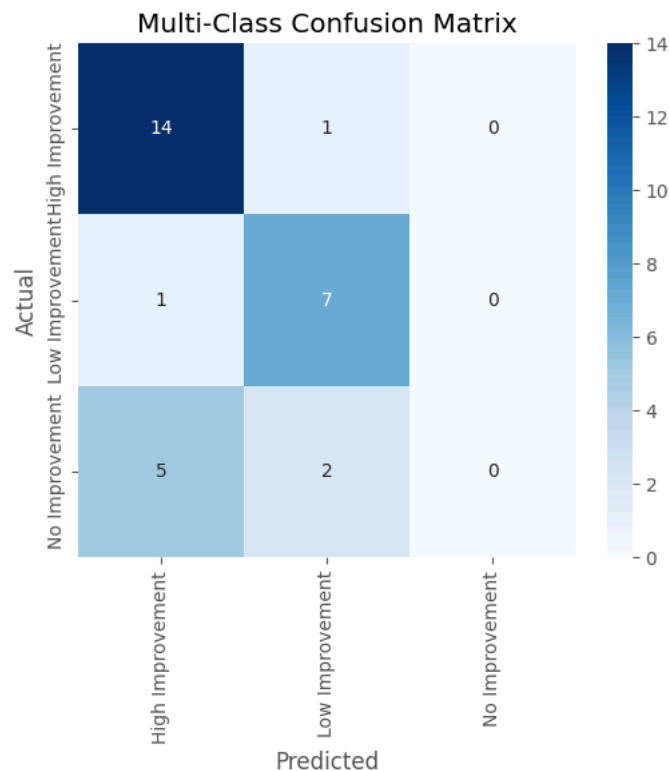
```

```
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



REGRESSION MODEL Step 19: Predict Improvement Value

```
y_reg = data[
    "Performance_Improvement"
]

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X,
    y_reg,
    test_size=0.2,
    random_state=42
)

rf_reg = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

rf_reg.fit(
    X_train_reg,
    y_train_reg
)

pred_reg = rf_reg.predict(
    X_test_reg
)
```

Step 20: Regression Evaluation

```
mae = mean_absolute_error(
    y_test_reg,
    pred_reg
)
```

```

r2 = r2_score(
    y_test_reg,
    pred_reg
)

print(
    "MAE:",
    round(mae,2)
)

print(
    "R2 Score:",
    round(r2,3)
)

```

MAE: 8.01
R² Score: 0.036

Step 21: Actual vs Predicted Plot

```

plt.figure(figsize=(8,6))

plt.scatter(
    y_test_reg,
    pred_reg,
    s=100,
    alpha=0.7
)

line_min = min(
    y_test_reg.min(),
    pred_reg.min()
)

line_max = max(
    y_test_reg.max(),
    pred_reg.max()
)

plt.plot(
    [line_min, line_max],
    [line_min, line_max],
    '--',
    linewidth=2,
    label='Ideal Prediction'
)

plt.text(
    0.05,
    0.95,
    f'R2 = {r2:.3f}',
    transform=plt.gca().transAxes
)

plt.xlabel(
    'Actual Improvement'
)

plt.ylabel(
    'Predicted Improvement'
)

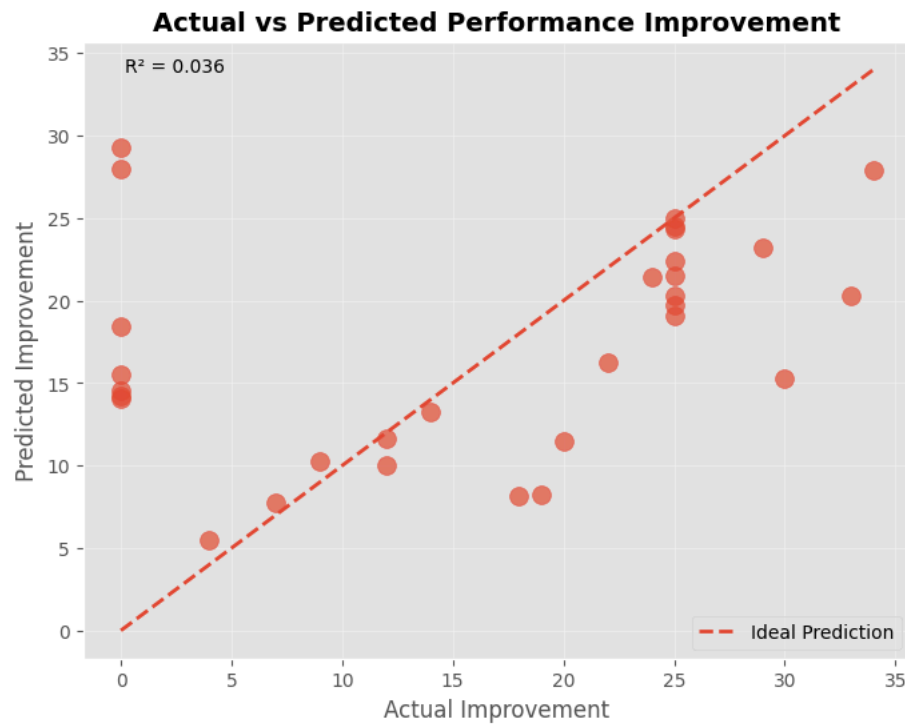
plt.title(
    'Actual vs Predicted Performance Improvement',
    fontsize=14,
    fontweight='bold'
)

plt.legend()

plt.grid(alpha=0.3)

plt.show()

```



Step 22: Project KPIs

```
total_pages = len(data)

improved_pages = (
    data["Performance_Improvement"] > 0
).sum()

improvement_rate = round(
    improved_pages /
    total_pages * 100,
    2
)

print(
    "Total Pages:",
    total_pages
)

print(
    "Improved Pages:",
    improved_pages
)

print(
    "Improvement Rate:",
    improvement_rate,
    "%"
)
```

```
Total Pages: 150
Improved Pages: 120
Improvement Rate: 80.0 %
```

Start coding or [generate](#) with AI.

```
Total Pages: 150
Improved Pages: 120
Improvement Rate: 80.0 %
```

```
kpi_data = pd.DataFrame({
    "Metric": [
        "Total Pages",
        "Improved Pages"
    ],
    "Value": [
        150,
        120
    ]
})
```

```

    "Value": [
        total_pages,
        improved_pages
    ]
})

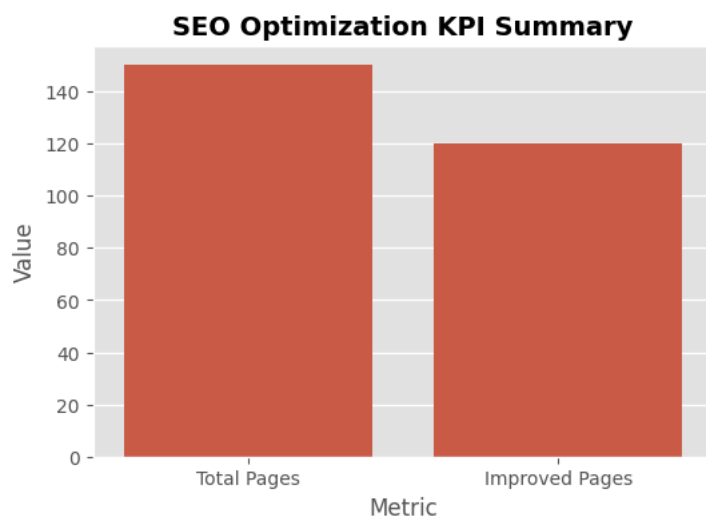
plt.figure(figsize=(6,4))

sns.barplot(
    data=kpi_data,
    x="Metric",
    y="Value"
)

plt.title(
    "SEO Optimization KPI Summary",
    fontsize=14,
    fontweight="bold"
)

plt.show()

```



Dashboard

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

plt.style.use("ggplot")

# =====
# KPI VALUES
# =====

total_pages = len(data)

improved_pages = (
    data["Performance_Improvement"] > 0
).sum()

improvement_rate = round(
    improved_pages /
    total_pages * 100,
    2
)

# R²

r2 = r2_score(
    y_test_reg,
    pred_reg
)

# =====

```

```

# FEATURE IMPORTANCE
# =====

importance = pd.DataFrame({
    "Feature": features,
    "Importance": rf_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

# =====
# SEO STATUS
# =====

before_counts = data[
    "SEO Issue Status_before"
].value_counts()

after_counts = data[
    "SEO Issue Status_after"
].value_counts()

comparison = pd.DataFrame({
    "Before": before_counts,
    "After": after_counts
})

# =====
# DASHBOARD
# =====

fig = plt.figure(
    figsize=(22,14)
)

gs = fig.add_gridspec(
    4,
    4,
    hspace=0.5,
    wspace=0.4
)

fig.suptitle(
    "SEO Performance & Machine Learning Dashboard",
    fontsize=24,
    fontweight="bold"
)

# =====
# KPI CARDS
# =====

kpis = [

    ("Total Pages", total_pages),

    ("Improved Pages", improved_pages),

    ("Improvement Rate", f"{improvement_rate}%"),

    ("R2 Score", round(r2,3))

]

for i, (title, value) in enumerate(kpis):

    ax = fig.add_subplot(gs[0, i])

    ax.axis("off")

    ax.text(
        0.5,
        0.6,
        value,

```



```

        fontsize=28,
        fontweight="bold",
        ha="center"
    )

    ax.text(
        0.5,
        0.3,
        title,
        fontsize=13,
        ha="center"
    )

# =====
# SEO STATUS COMPARISON
# =====

ax1 = fig.add_subplot(gs[1, 0:2])

comparison.plot(
    kind="bar",
    ax=ax1
)

ax1.set_title(
    "SEO Status Comparison",
    fontsize=14,
    fontweight="bold"
)

ax1.set_ylabel("Count")

# =====
# IMPROVEMENT DISTRIBUTION
# =====

ax2 = fig.add_subplot(gs[1, 2:4])

sns.histplot(
    data["Performance_Improvement"],
    bins=15,
    kde=True,
    ax=ax2
)

ax2.set_title(
    "Performance Improvement Distribution",
    fontsize=14,
    fontweight="bold"
)

# =====
# BEFORE VS AFTER
# =====

ax3 = fig.add_subplot(gs[2, 0:2])

ax3.scatter(
    data["Performance_Score_before"],
    data["Performance_Score_after"],
    alpha=0.8,
    s=70
)

ax3.plot(
    [0,100],
    [0,100],
    '-_'
)

ax3.set_title(
    "Before vs After Performance",
    fontsize=14,
    fontweight="bold"
)

ax3.set_xlabel("Before")

```

```

ax3.set_ylabel("After")

# =====
# FEATURE IMPORTANCE
# =====

ax4 = fig.add_subplot(gs[2, 2:4])

sns.barplot(
    data=importance,
    x="Importance",
    y="Feature",
    ax=ax4
)

ax4.set_title(
    "Random Forest Feature Importance",
    fontsize=14,
    fontweight="bold"
)

# =====
# CLASS DISTRIBUTION
# =====

ax5 = fig.add_subplot(gs[3, 0:2])

sns.countplot(
    data=data,
    x="Improvement_Class",
    ax=ax5
)

ax5.set_title(
    "Improvement Class Distribution",
    fontsize=14,
    fontweight="bold"
)

# =====
# ACTUAL VS PREDICTED
# =====

ax6 = fig.add_subplot(gs[3, 2:4])

ax6.scatter(
    y_test_reg,
    pred_reg,
    alpha=0.8,
    s=70
)

line_min = min(
    y_test_reg.min(),
    pred_reg.min()
)

line_max = max(
    y_test_reg.max(),
    pred_reg.max()
)

ax6.plot(
    [line_min, line_max],
    [line_min, line_max],
    '--',
    linewidth=2
)

ax6.set_title(
    "Actual vs Predicted",
    fontsize=14,
    fontweight="bold"
)

ax6.set_xlabel("Actual")
ax6.set_ylabel("Predicted")

```

ax6.txtf